

ASP.NET Core Out of Process Hosting Model

Back to: [ASP.NET Core Web API Tutorials](#)

ASP.NET Core Out-of-Process Hosting Model

In ASP.NET Core, hosting models define how an application runs and interacts with a web server. Broadly, there are two models:

- **In-Process Hosting:** The app runs directly inside the IIS worker process.
- **Out-of-Process Hosting:** The app runs in a separate process, and IIS acts as a reverse proxy.

Now, we will focus on the **Out-of-Process Hosting Model**, its architecture, configuration, workflow, and practical analogies. Understanding this model is crucial for trainers and developers because it highlights how IIS and Kestrel collaborate to provide scalability, security, and performance in ASP.NET Core applications.

What is Out-of-Process Hosting?

In Out-of-Process Hosting, your ASP.NET Core application does not run directly inside the IIS worker process. Instead, **two web servers work together**:

IIS (External Web Server)

- Handles client connections, SSL/TLS termination, logging, and URL rewriting.
- Acts as the **gateway**, protecting your app from direct internet exposure.

Kestrel (Internal Web Server)

- Runs your middleware pipeline, controllers, and business logic.
- Focused on performance and executing application code.

This separation of responsibilities ensures robustness; **IIS handles outside concerns, while Kestrel handles inside concerns**.

Real-World Analogy

Think of an **airport system**:

- **IIS is the Airport Terminal** – manages check-in, security checks, and boarding.
- **Kestrel is the Airplane** – takes passengers (requests) to their destination.

The terminal doesn't fly planes, and the plane doesn't check tickets. Together, they provide a secure, efficient journey.

How to Configure the Out-of-Process Hosting in an ASP.NET Core Application?

Configuring **Out-of-Process Hosting** in an ASP.NET can be done in two ways: either by editing the project file directly or by using the Visual Studio GUI.

Method 1: Editing Project File

In this approach, you need to add the **<AspNetCoreHostingModel>** element inside your project's .csproj file and set its value to **OutOfProcess**. This property instructs the .NET runtime to run the application outside the IIS worker process, instead in a separate process using the **Kestrel server**.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>
</Project>
```

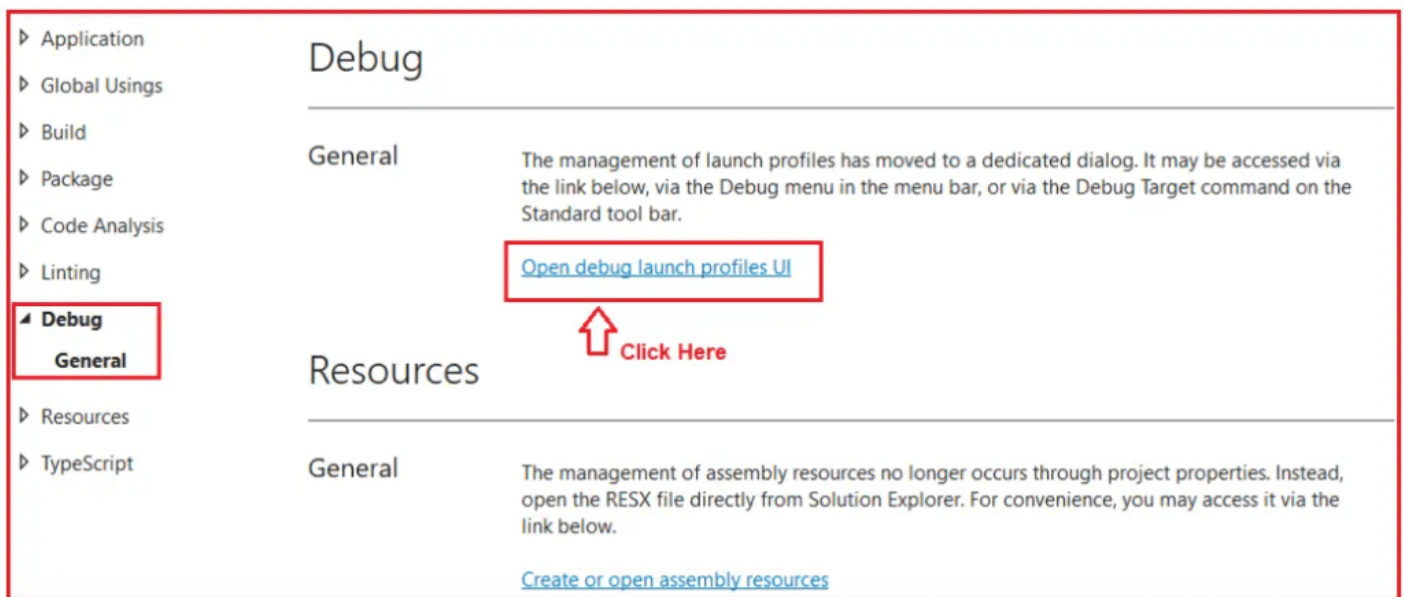
With the above changes in place, when you build and publish the project, this configuration ensures IIS will only forward requests to Kestrel, and Kestrel will execute your application code.

Please consider this as instructing a restaurant kitchen on whether to cook food **in-house** or **outsource it**. If you set it to **InProcess**, IIS cooks everything inside the restaurant kitchen. But when you set it to **OutOfProcess**, IIS takes the orders and sends them to a separate food vendor (Kestrel), who actually cooks the meals. IIS then delivers the meals back to the customers.

Method 2: Using Visual Studio GUI

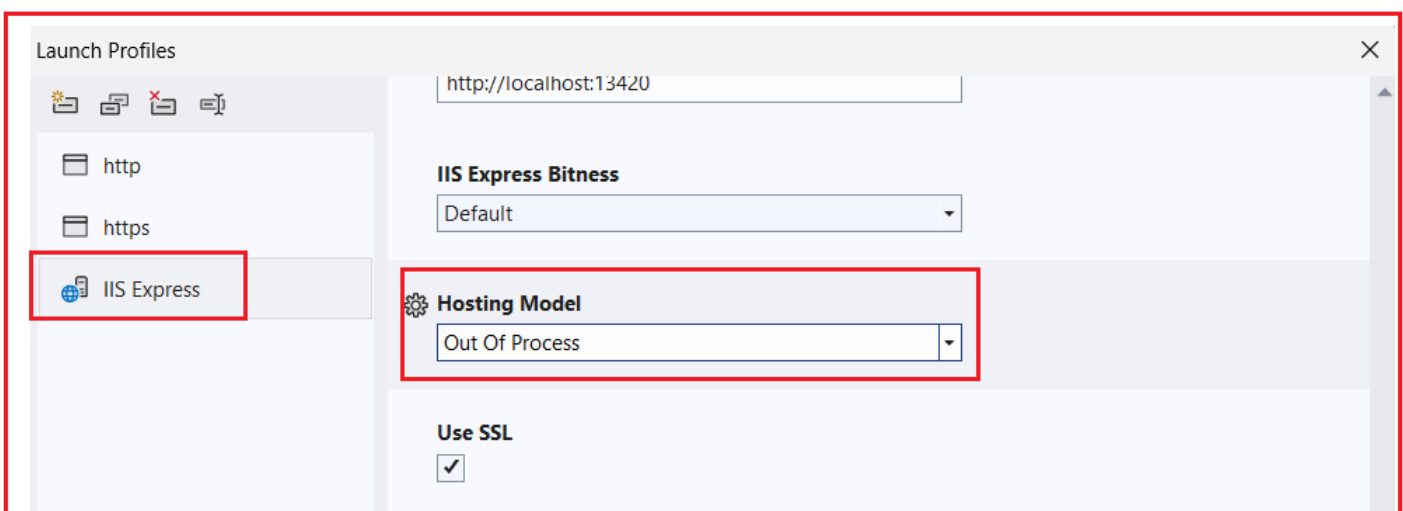
If you don't want to manually edit the .csproj file, Visual Studio provides a user-friendly option.

- First, right-click on the project in **Solution Explorer** and select **Properties**.
- From the Project Properties window, open the **Debug** tab.
- Click the **"Open debug launch profiles UI"** button, as shown in the image below.



This opens the Launch Profile configuration window.

- Select the **IIS Express** profile, and scroll down until you find the “**Hosting Model**” drop-down.
- From the list, select **OutOfProcess** as shown in the image below.



How Do We Use Out-of-Process Hosting in ASP.NET Core?

When discussing **Out-of-Process Hosting in ASP.NET Core**, it's essential first to understand when this model is applicable.

- If you run your application directly with the **HTTP** or **HTTPS** profiles in Visual Studio (for example, by pressing F5 without choosing IIS Express), your application runs **only on Kestrel**.
- In that case, there is no concept of **InProcess** or **OutOfProcess**, because IIS isn't involved at all.
- Kestrel is the single server responsible for receiving the requests and executing your application code.

However, when you run your application using the **IIS Express launch profile in Visual Studio**, the hosting model takes effect. IIS Express will act as the **Front-Facing Web Server**, and your ASP.NET Core app can either run inside **IIS (In-Process)** or in a separate process with **Kestrel (Out-of-Process)**.

If configured for **Out-of-Process hosting**, IIS does not execute your application directly. Instead, it acts as a **Reverse Proxy**: it accepts incoming client requests, forwards them to Kestrel, and then passes Kestrel's response back to the client.

- **Running with Kestrel only (HTTP/HTTPS profiles)**: Kestrel is both the gatekeeper (receiving requests) and the executor (running your app code).
- **Running with IIS Express (Out-of-Process hosting)**: IIS is the gatekeeper, and Kestrel is the executor. IIS forwards requests to Kestrel, Kestrel processes them, and IIS sends the response back.

Real-world analogy:

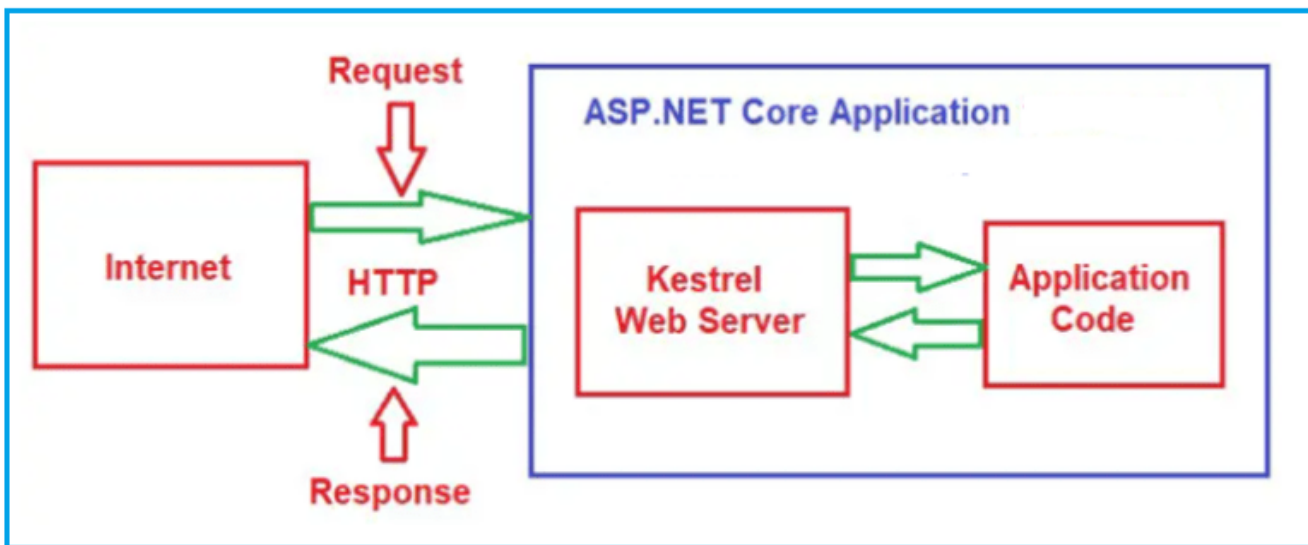
Imagine a company office.

- If you call the **Team Directly** (Kestrel with the HTTP/HTTPS profile), the team answers the call, listens to your request, and does the work, all by themselves.
- If you call through the **Company Receptionist (IIS)** in **Out-of-Process mode**, the receptionist takes your call, notes your request, and then transfers it to the right team (Kestrel). Once the team finishes the work, the receptionist gives the result back to you. The receptionist doesn't do the actual work. They manage communication and ensure everything flows smoothly.

Kestrel as the Internet-Facing Web Server

When you run an ASP.NET Core application **directly using HTTP or HTTPS profiles**, Kestrel takes on the role of the **Internet-Facing Web Server**. In this mode, no IIS is acting as a front-end reverse proxy. Instead, Kestrel is responsible for doing everything: it receives incoming HTTP requests from the client (such as a browser), processes them by executing your application code, and then sends the response back to the client. This setup is simple and lightweight, making it especially useful in **Development Environments** or in scenarios where you don't need the advanced features of a full-fledged web server, such as IIS.

In this case, Kestrel is both the **gatekeeper** (accepting requests directly from the internet) and the **executor** (running your ASP.NET Core middleware and controllers). However, since Kestrel is exposed directly to the internet, it doesn't provide enterprise-level features such as advanced request filtering, URL rewriting, or built-in load balancing. That's why in **production environments**, Kestrel is usually paired with IIS, Apache, or Nginx, which handle those additional responsibilities.



Worker Process in Kestrel

The worker process that runs your application depends on how you **deploy** it:

- **Framework-Dependent Deployment:** If your app is published in a way that relies on the .NET runtime installed on the server, then the process that runs it will be **dotnet.exe**. In this case, when you start the app, you're really launching the .NET runtime, and the runtime loads your application DLL into memory and executes it.
- **Self-Contained Deployment:** If your app is published as self-contained, it includes its own copy of the .NET runtime. That means you don't rely on the host machine having .NET installed. In this case, the process name will be the **name of your application executable**. For example, if your project is called **MyFirstWebAPIProject**, the process running on Windows will appear as **MyFirstWebAPIProject.exe**.

Run the application using HTTP/HTTPS Launch Profile:

Let us prove this. First, modify the TestController as follows:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Hosting.Server;
using System.Diagnostics;

namespace MyFirstWebAPIProject.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class TestController : ControllerBase
    {
        // The IServer interface represents the actual server implementation
        // (IISHttpServer or KestrelServer)
        private readonly IServer _server;
```

```

        // Constructor Dependency Injection - ASP.NET Core injects the active
        IServer service
        public TestController(IServer server)
        {
            _server = server;
        }

        // GET: api/test/ping
        [HttpGet("ping")]
        public IActionResult Ping()
        {
            // Get the current process information (e.g., w3wp, iisexpress,
            dotnet)

            var proc = Process.GetCurrentProcess();

            // Return hosting environment details as JSON response
            return Ok(new
            {
                message = "Hosting Info",           // Custom message
                pid = Environment.ProcessId,         // Numeric process ID
                process = proc.ProcessName,          // Running process name
                serverType = _server.GetType().Name, // Hosting server type
                (IISHttpServer or KestrelServer)
                machine = Environment.MachineName    // Server machine name
            });
        }
    }
}

```

Now, run the application using either the http or https profiles and verify the worker process name. Now, hit the **api/Test/ping** endpoint, then you will see the following output. Here, it shows MyFirstWebAPIProject, indicating that the Kestrel Server is executing our application code.

Request URL

https://localhost:7191/api/Test/ping

Server response

Code	Details
200	Response body <pre>{ "message": "Hosting Info", "pid": 17692, "process": "MyFirstWebAPIProject", "serverType": "KestrelServerImpl", "machine": "LAPTOP-6P5NK25R" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Mon, 29 Sep 2025 04:14:44 GMT server: Kestrel</pre>

This proves that when we use the HTTP or HTTPS profile to launch our application in Visual Studio, Kestrel is the only Web Server that hosts our application, handles the incoming HTTP Requests, and executes the application code. In this case, Kestrel works as an Internet-facing Web Server.

Why do you see MyFirstWebAPIProject.exe instead of dotnet.exe?

When you press **F5** (or run with the HTTP/HTTPS profile) in Visual Studio, the IDE builds your project into an **output folder** (like bin\Debug\net8.0). What gets generated there depends on the type of build:

- Even though the project is a **Framework-Dependent Deployment (FDD)**, Visual Studio still produces an **executable file** (MyFirstWebAPIProject.exe on Windows).
- This .exe is basically a bootstrapper. Internally, it just calls the dotnet host, which loads the runtime and runs your project's .dll.

So, in **Task Manager**, you will see **MyFirstWebAPIProject.exe** because that's the process wrapper Visual Studio launches. You don't see the underlying **dotnet.exe** because the bootstrapper hides it.

In short:

- **dotnet.exe** = The generic runtime host (when you explicitly run dotnet MyApp.dll).
- **MyFirstWebAPIProject.exe** = The app-specific host executable that Visual Studio runs during development.

Running ASP.NET Core under dotnet.exe using a custom launch profile

By default, when you run an ASP.NET Core application from Visual Studio, it launches the project-specific executable generated for the application (for example, `MyFirstWebAPIProject.exe`). Even if the project is a Framework-Dependent Deployment (FDD), the .NET SDK creates a small bootstrap exe that internally calls `dotnet MyFirstWebAPIProject.dll`. Because Visual Studio runs this exe, what you see in Task Manager is your project's exe name, not `dotnet.exe`.

If you want to **see dotnet.exe as the active process**, you need to bypass that bootstrap exe and explicitly run your DLL using the dotnet host. This is where a **custom launch profile** in `launchSettings.json` comes in.

Creating a Custom DotNetLauncher Profile

You can add a new profile inside the profiles section of `launchSettings.json` as follows:

```
"DotNetLauncher": {
  "commandName": "Executable",
  "executablePath": "dotnet",
  "commandLineArgs": "MyFirstWebAPIProject.dll --urls
\"https://localhost:7191;http://localhost:5021\"",
  "workingDirectory": "$(ProjectDir)bin\\Debug\\net8.0",
  "launchBrowser": true,
  "launchUrl": "swagger",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
}
```

Here,

- “commandName”: “Executable” tells Visual Studio to run a custom executable instead of the default project exe.
- “executablePath”: “dotnet” ensures the .NET runtime host itself is launched.
- “commandLineArgs”: “MyFirstWebAPIProject.dll ...” tells dotnet.exe which DLL to load and on which URLs Kestrel should listen.
- “workingDirectory” points to the folder containing the DLL.

When you run this profile, Task Manager will now show **dotnet.exe** because that is the process actually executing your app.

Why does Swagger not auto-open in this profile

Now, run the application using the DotNetLauncher profile. Visual Studio normally respects “launchBrowser”: true and “launchUrl” for **Project** and **IIS Express** profiles. In those cases, VS knows it’s running a web application and automatically launches a browser at the configured URL.

However, for **Executable** profiles, Visual Studio behaves differently: it simply starts the process you told it to run (dotnet.exe) and doesn’t treat it as a web app in the same way. As a result, even if you set launchBrowser: true, the browser doesn’t open automatically. This is expected behaviour, not a bug.

How to test Swagger in this setup

Once your app is running under the DotNetLauncher profile, you can simply open your browser and manually navigate to: **https://localhost:7191/swagger**

Here, you will see the Swagger UI, just as you would under the normal Project or IIS Express profile. The difference is that now, when you check Task Manager, the **process name is dotnet.exe** instead of your project’s executable. Now, testing the endpoint as shown in the image below. Now, it is displaying the process name as ‘dotnet’.

Request URL

`https://localhost:7191/api/Test/ping`

Server response

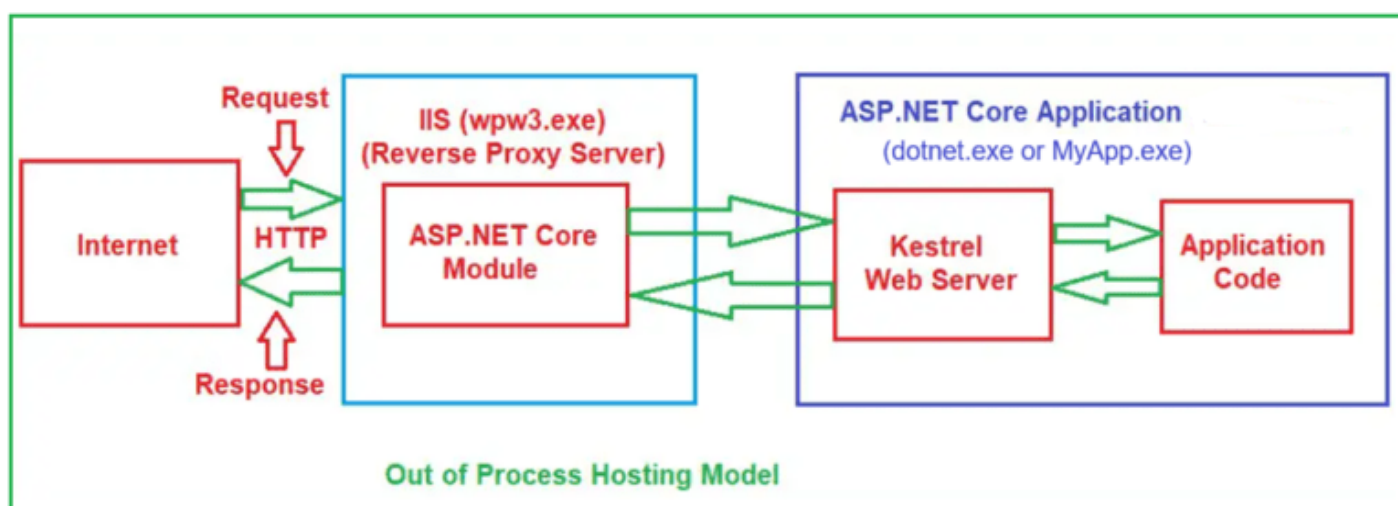
Code	Details
200	Response body <pre>{ "message": "Hosting Info", "pid": 23872, "process": "dotnet", "serverType": "KestrelServerImpl", "machine": "LAPTOP-6P5NK25R" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Mon, 29 Sep 2025 06:01:57 GMT server: Kestrel</pre>

Key points to remember

- Visual Studio usually launches the project's generated exe → process name shows as your project name.
- To run under dotnet.exe, you must create a custom Executable profile that calls dotnet <YourApp>.dll.
- LaunchBrowser doesn't auto-work for Executable profiles; you need to open Swagger manually.
- Once running, functionality is identical; the only difference is the process hosting your app.

Out-of-Process Hosting Model in ASP.NET Core

In the **Out-of-Process Hosting Model**, the ASP.NET Core application does not run inside IIS's worker process. Instead, IIS plays the role of a **Reverse Proxy**, while the actual application code runs in a separate **Kestrel Process**. This separation allows IIS to handle internet-facing concerns such as SSL, security, and request filtering, while Kestrel focuses on executing the application logic. For a better understanding, please refer to the following diagram.



Let us understand how the Out-of-Process Hosting Model works.

Step 1: Client Sends Request

The process begins when a user interacts with your application, for example, typing a URL into the browser or calling an API. The browser or client sends an HTTP/HTTPS request over the Internet. This request first arrives at **IIS**, the external web server configured to host the application.

Step 2: IIS as the Reverse Proxy

When IIS receives the request, it doesn't process the ASP.NET Core pipeline itself (because the app is not running inside its worker process). Instead, IIS works as a **Reverse Proxy Server**, whose job is to forward the request to the actual application process running on Kestrel. IIS uses a special bridge called the **ASP.NET Core Module (ANCM)** to communicate with Kestrel.

Step 3: ASP.NET Core Module (ANCM)

ANCM is a native IIS module that manages the lifecycle of the ASP.NET Core application:

- It checks whether the Kestrel process is already running.
- If Kestrel is not running, ANCM **starts the application process** (launching dotnet.exe MyApp.dll or MyApp.exe in case of a self-contained deployment).
- Once Kestrel is alive, ANCM forwards the HTTP request to Kestrel over a local port (HTTP-only, not HTTPS).

This ensures that IIS and Kestrel can work together seamlessly, without requiring you to start or monitor the app manually.

Step 4: Kestrel Receives Request

Kestrel is the internal web server bundled with ASP.NET Core. Its role is to **accept forwarded requests from ANCM** and pass them into the ASP.NET Core request pipeline. Kestrel doesn't worry about SSL termination, load balancing, or rewriting; it simply focuses on handling the HTTP request and delivering it to the application code.

Step 5: ASP.NET Core Application Code Executes

Once Kestrel hands the request to the ASP.NET Core pipeline, the application logic takes over:

- **Middleware** may perform cross-cutting tasks such as logging, error handling, CORS checks, or authentication.
- **Authentication and Authorization** steps verify the user's identity and permissions.
- **Routing** determines which controller, Razor Page, or endpoint should handle the request.
- **Business logic** executes, possibly involving database queries, external API calls, or internal calculations.

After all middleware and business logic have been applied, the application produces a response (for example, JSON data for an API, or HTML content for a web page).

Step 6: Kestrel Sends Response Back

The response generated by the application code is handed back to Kestrel. Kestrel's job is to forward this completed response back through the hosting pipeline.

Step 7: ANCM Relays the Response

Kestrel passes the response to the ASP.NET Core Module (ANCM). At this stage, ANCM does not modify or process the response; it simply returns it to IIS.

Step 8: IIS Prepares the Response for the Client

IIS receives the application response from ANCM and gets it ready for delivery back to the requesting client. IIS may apply additional features here, such as response compression, caching, or final security checks.

Step 9: Client Receives the Response

Finally, the response travels back over the internet to the client (for example, the user's browser). The browser then displays the content, whether it is a rendered web page, API data, or any other response returned by the ASP.NET Core application.

Why these matters?

This layered approach gives you the best of both worlds:

- IIS handles tasks like SSL termination, Windows Authentication, request filtering, and managing thousands of simultaneous connections.
- Kestrel stays lightweight and focused, efficiently executing the .NET Core application code.

The separation also enables ASP.NET Core apps to work consistently across platforms. On Windows, you might pair Kestrel with IIS, while on Linux, you might pair it with Nginx or Apache; however, the flow remains the same.

Running the Application using the IIS Express Launch Profile

Now, run the application using the IIS Express profile. As shown in the image below, you will see the worker process as your project name. You can also verify in the taskbar that the IIS Server is running.

Request URL

https://localhost:44318/api/Test/ping

Server response

Code	Details
200	Response body <pre>{ "message": "Hosting Info", "pid": 40972, "process": "MyFirstWebAPIProject", "serverType": "KestrelServerImpl", "machine": "LAPTOP-6P5NK25R" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Mon, 29 Sep 2025 04:28:58 GMT server: Kestrel x-powered-by: ASP.NET</pre>

With the Out-of-Process Hosting Model, when we use IIS Express as the launch profile to run the application, **IIS Express** acts as the **Reverse Proxy Server (Internet-facing server)**, and **Kestrel** acts as the **Internal Web Server**.

Now, with the Out-of-Process Hosting Model, IIS Express (IIS) receives the incoming HTTP request and then forwards it to the Kestrel Web Server for processing. The Kestrel Web Server processes the request, generates the response, and sends it back to IIS Express, which in turn sends the response back to the client, i.e., to the browser.

Out-of-Process Hosting in ASP.NET Core separates responsibilities: **IIS** acts as a reverse proxy (handling security, SSL, and logging), while **Kestrel** executes your app code. This model ensures **scalability, security, and cross-platform consistency**, making it the preferred hosting approach for production-ready applications.



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information